



UNIVERSIDADE FEDERAL DA BAHIA
ENGENHARIA DE SOFTWARE I - MATA62 - T01

FELIPE SILVA DA ANUNCIAÇÃO
JAEDSON BARBOSA MACEDO
LUCAS PEREIRA RAPOZO
NADSON DE SANTANA SOUSA

TRABALHO I: ESTILO ARQUITETURAL MICROKERNEL

SALVADOR
2026

TRABALHO I: ESTILO ARQUITETURAL MICROKERNEL

Análise dos sistemas open source Visual Studio Code e OBS Studio

Trabalho I apresentado à disciplina de Engenharia de Software I, do curso de Bacharelado em Ciência da Computação da Universidade Federal da Bahia (UFBA), como requisito de avaliação parcial.

Docente: Prof. Dr. Eduardo Almeida.

Equipe 03:

Felipe Silva da Anunciação;

Jaedson Barbosa Macedo;

Lucas Pereira Rapozo;

Nadson de Santana Sousa.

SUMÁRIO

SUMÁRIO.....	2
1. VISUAL STUDIO CODE (VS CODE).....	4
1.1 DECISÕES ARQUITETURAIS.....	4
1.2 DIAGRAMA DE MÓDULOS DA ARQUITETURA.....	5
1.3 CARACTERÍSTICAS ARQUITETURAIS E TÁTICAS.....	6
1.4 CARACTERÍSTICAS ARQUITETURAIS.....	8
1.5 CARACTERÍSTICAS ARQUITETURAIS - TOP 4.....	10
1.6 COMPONENTES CANDIDATOS.....	11
1.7 DIAGRAMA DE COMPONENTES DA ARQUITETURA.....	13
2. OBS STUDIO.....	13
2.1 CARACTERÍSTICAS ARQUITETURAIS.....	13
2.2 CARACTERÍSTICAS ARQUITETURAIS - TOP 4.....	17
2.3 LISTA DE COMPONENTES CANDIDATOS.....	20
2.4 DIAGRAMA DE COMPONENTES DA ARQUITETURA.....	23
2.5 DECISÕES ARQUITETURAIS.....	24
2.6 DIAGRAMA DE MÓDULOS DA ARQUITETURA.....	26
2.7 TÁTICAS PARA CARACTERÍSTICAS.....	27
REFERÊNCIAS.....	29

1. VISUAL STUDIO CODE (VS CODE)

Repositório do GitHub: [microsoft/vscode: Visual Studio Code](https://github.com/microsoft/vscode)

1.1 DECISÕES ARQUITETURAIS

As decisões arquiteturais formam as regras para a construção dos sistemas. Um Architecture Decision Record (ADR) captura uma única decisão arquitetural e sua justificativa, sendo essencial para documentar o ciclo de vida do projeto. A estrutura adotada contém Título, Status, Contexto, Decisão e Consequências.

Decisão 1: Adoção do Estilo Microkernel

- **Título:** ADR-001 Adoção da Arquitetura Baseada em Plugins (Microkernel).
- **Status:** Aceito.
- **Contexto:** O sistema necessita ser altamente extensível sem causar inchaço no núcleo principal (*Core*), evitando a degradação estrutural e o antipadrão *Big Ball of Mud*.
- **Decisão:** Foi definida a utilização da arquitetura Microkernel (ou *Plugin-Based Architecture*), composta por um sistema central (Core System) e múltiplos plugins acopláveis.
- **Consequências:** Esta decisão promove um forte desacoplamento e facilita a manutenção a longo prazo, definindo um vocabulário claro de componentes.

Decisão 2: Isolamento de Extensões

- **Título:** ADR-002 Isolamento de Extensões via Extension Host.
- **Status:** Aceito.
- **Contexto:** Códigos de terceiros podem conter bugs, prejudicando a confiabilidade do sistema e bloqueando a interface principal do usuário.
- **Decisão:** Nós iremos isolar o código de customização (plugins) executando-os em um processo totalmente separado, chamado *Extension Host*, interagindo com o núcleo via chamadas remotas ou eventos assíncronos.

- **Consequências:** Esta tática garante que a falha de um plugin não derrube a interface gráfica do editor, ainda que o custo de comunicação remota introduza uma latência que não é zero.

Decisão 3: Padronização de Serviços de Linguagem

- **Título:** ADR-003 Implementação do Language Server Protocol (LSP).
- **Status:** Aceito.
- **Contexto:** Suportar múltiplas linguagens exige o desenvolvimento de lógicas complexas que não devem poluir o ambiente central (Core System).
- **Decisão:** Nós iremos utilizar o particionamento de domínio para delegar a análise sintática de código a serviços distribuídos (Language Servers), comunicando-se de forma padronizada.
- **Consequências:** Aumenta exponencialmente a reutilização, visto que os provedores das linguagens podem usar o mesmo servidor em outras ferramentas, configurando um claro "trade-off" positivo para a visão do negócio.

Decisão 4: Framework de Interface Multiplataforma

- **Título:** ADR-004 Utilização do Framework Electron.
- **Status:** Aceito.
- **Contexto:** A organização precisava construir o sistema para Windows, Linux e macOS rapidamente, necessitando equilibrar restrições orçamentárias e velocidade (Business Drivers).
- **Decisão:** Nós iremos encapsular tecnologias Web (HTML/JS/CSS) usando o Electron como casca (wrapper) para a aplicação desktop.
- **Consequências:** Reduz radicalmente a necessidade de múltiplos times de desenvolvimento nativo, em troca de um maior consumo de recursos de máquina (trade-off de performance).

1.2 DIAGRAMA DE MÓDULOS DA ARQUITETURA

O diagrama a seguir ilustra a separação física e lógica dos principais módulos do VS Code, evidenciando o isolamento entre a interface de usuário, o gerenciador de extensões e os plugins de terceiros.

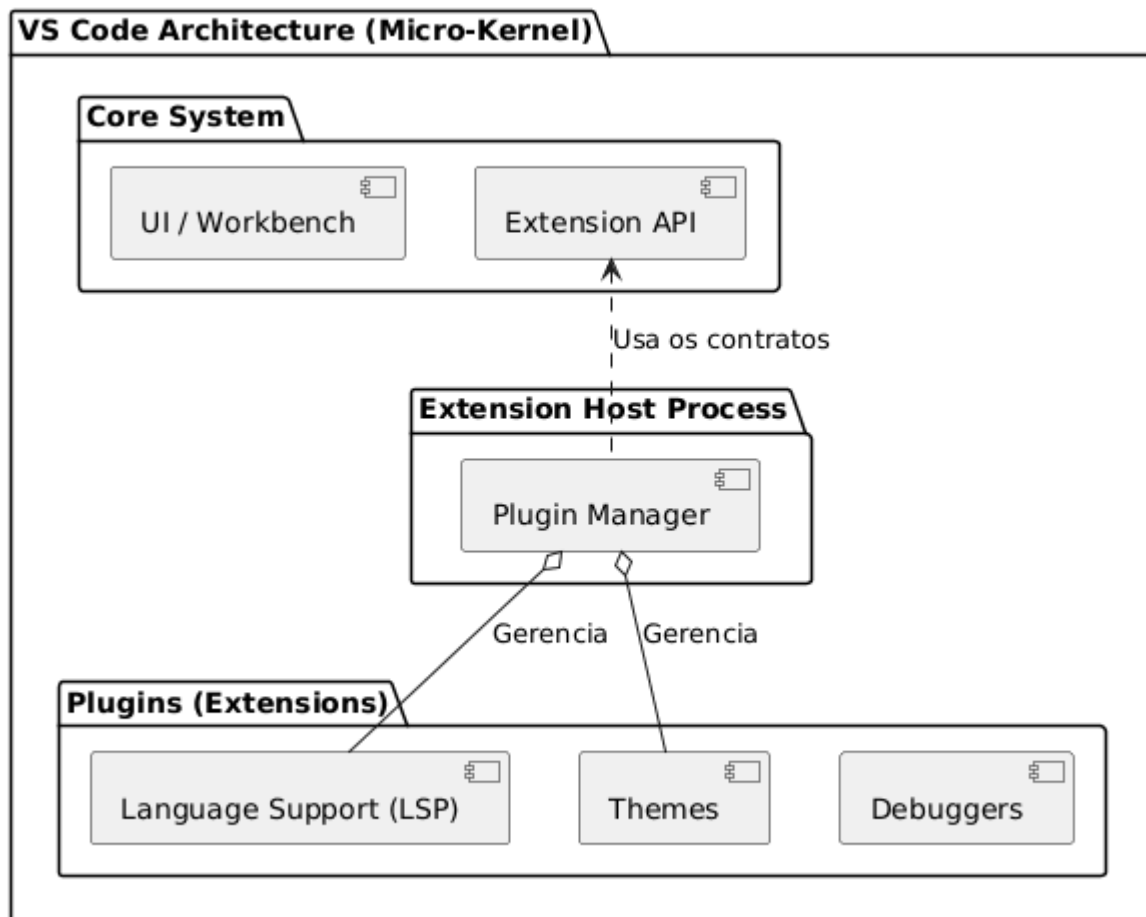


Diagrama de Módulos do VS Code

Fonte: Elaboração própria (2026).

1.3 CARACTERÍSTICAS ARQUITETURAIS E TÁTICAS

Com base na arquitetura estabelecida e nas categorias de características arquiteturais, as seguintes táticas foram aplicadas para satisfazer os requisitos do sistema:

- **Categoria Estrutural: Extensibilidade e Alavancagem/Reuso**
 - *Definição base:* Facilidade de conectar novas peças e aproveitar componentes comuns.

- *Tática Aplicada:* Utilizou-se o particionamento de componentes através do estilo Microkernel e o particionamento de domínio usando o *Language Server Protocol* (LSP).
- *Objetivo:* Permitir a adição de plugins (extensões) ilimitados sem alterar o núcleo do sistema, e reaproveitar os servidores de linguagem em outras ferramentas externas, reduzindo drasticamente o acoplamento.
- **Categoria Operacional: Tolerância a Falhas e Confiabilidade/Segurança**
 - *Definição base:* O sistema opera conforme planejado mesmo com falhas, evitando que bugs de terceiros afetem o núcleo.
 - *Tática Aplicada:* Isolamento de processos (*Process Isolation*) por meio da criação do *Extension Host*.
 - *Objetivo:* Garantir que códigos customizados de extensões sejam executados em um processo separado. Caso uma extensão gere consumo excessivo de CPU ou *crashes*, a anomalia não derrubará a interface gráfica principal (*UI/Workbench*).
- **Categoria Operacional: Desempenho (Performance)**
 - *Definição base:* Otimização dos tempos de resposta e capacidade de processamento da interface.
 - *Tática Aplicada:* Troca de mensagens assíncronas (*Asynchronous Messaging*) entre o *Core System* e os provedores de extensão.
 - *Objetivo:* Impedir que operações lentas, como compilação de código pela extensão, bloqueiem a *thread* principal do usuário, mantendo a renderização de tela e a digitação em tempo real fluida.
- **Categoria Processo: Agilidade**
 - *Definição base:* Capacidade de responder e adaptar-se rapidamente a mudanças.
 - *Tática Aplicada:* Desacoplamento da arquitetura por meio da padronização da *Extension API*.
 - *Objetivo:* Facilitar a evolução independente tanto do núcleo do editor quanto do ecossistema de extensões de terceiros, permitindo entregas

contínuas e rápidas sem necessidade de uma recompilação completa do produto.

1.4 CARACTERÍSTICAS ARQUITETURAIS

Característica: Extensibilidade

- **Título:** ADR-001 — Extensibilidade por meio de um Núcleo Pequeno e Extensões Baseadas em API Pública.
- **Status:** Aceito.
- **Contexto:** Um editor de código precisa atender múltiplas linguagens, ferramentas, fluxos de depuração, provedores de controle de versão e necessidades específicas de diferentes comunidades. Concentrar todas essas capacidades no núcleo tornaria o sistema rígido e difícil de evoluir.
- **Decisão:** Adotar uma arquitetura microkernel em que o núcleo oferece serviços e contratos de extensão, enquanto funcionalidades especializadas são implementadas como extensões. As extensões declaram comandos, linguagens, depuradores, temas, visualizações e outras contribuições por meio do arquivo `package.json`.
- **Consequências:** Novas funcionalidades podem ser criadas e distribuídas sem alteração direta no núcleo. Em contrapartida, a API de extensões passa a ser um contrato crítico de compatibilidade, documentação e governança.
- **Evidências:** O diretório `extensions/` contém extensões embarcadas, incluindo suporte a Git, TypeScript e Markdown. O contrato público está em `src/vscode-dts/vscode.d.ts`, enquanto os Contribution Points descrevem as contribuições declaradas no manifesto da extensão.

Característica: Desempenho

- **Título:** ADR-002 — Ativação Sob Demanda de Extensões para Preservar a Responsividade.
- **Status:** Aceito.

- **Contexto:** Usuários podem instalar dezenas ou centenas de extensões. Carregar todas durante a inicialização aumentaria o tempo de abertura do editor e o consumo de memória, prejudicando a experiência de uso.
- **Decisão:** Ativar extensões por eventos, como abertura de uma linguagem específica, execução de um comando, início de uma sessão de depuração ou acesso a uma visualização. A extensão só deve ser carregada quando sua funcionalidade for necessária.
- **Consequências:** A inicialização tende a ser mais rápida e o custo de execução é distribuído ao longo do uso. Por outro lado, extensões precisam declarar corretamente seus eventos de ativação para evitar carregamento tardio ou ativação excessiva.
- **Evidências:** A documentação de Activation Events descreve eventos como `onLanguage`, `onCommand`, `onDebug` e `onView`. O comando **Developer: Startup Performance** permite inspecionar o impacto da inicialização.

Característica: Confiabilidade e Tolerância a Falhas

- **Título:** ADR-003 — Isolamento do Host de Extensões para Conter Falhas de Plugins.
- **Status:** Aceito.
- **Contexto:** Extensões executam código de terceiros e podem falhar por erros de implementação, dependências incompatíveis ou alto consumo de recursos. Uma falha desse tipo não deve, idealmente, encerrar a interface principal do editor nem causar perda do trabalho em edição.
- **Decisão:** Executar extensões em um processo denominado **Extension Host**, separado da interface principal. Quando esse processo falha, o VS Code registra o evento, pode tentar reiniciá-lo e disponibiliza recursos de diagnóstico para identificar a extensão problemática.
- **Consequências:** Falhas são mais bem contidas e o editor pode recuperar parte das funcionalidades sem fechar toda a janela. Como custo, a comunicação entre o núcleo e as extensões exige protocolos, serialização de dados e mecanismos adicionais de supervisão.

- **Evidências:** O código de [abstractExtensionService.ts](#) trata o encerramento inesperado do Extension Host. O recurso **Extension Bisect** ajuda a localizar extensões que causam problemas por eliminação progressiva.

Característica: Manutenibilidade

- **Título:** ADR-004 — Organização em Camadas e Regras de Dependência para Manutenibilidade.
- **Status:** Aceito.
- **Contexto:** O VS Code é um projeto grande, multiplataforma e aberto a contribuições externas. Sem limites claros entre componentes, mudanças locais poderiam criar dependências indevidas e ampliar o risco de regressões.
- **Decisão:** Organizar o código em camadas, como base, platform, editor e workbench, estabelecendo que camadas de nível superior podem depender das inferiores, mas não o contrário. Complementar a organização com regras automatizadas de lint e revisão de código.
- **Consequências:** O código se torna mais previsível, mais simples de localizar e menos propenso a acoplamentos circulares. A contrapartida é que alterações entre camadas exigem mais planejamento e, às vezes, a criação de abstrações adicionais.
- **Evidências:** A wiki [Source Code Organization](#) descreve a divisão arquitetural. O repositório contém regras de qualidade e validações automatizadas para preservar essas fronteiras.

Característica: Portabilidade

- **Título:** ADR-005 — Abstração por Ambiente de Execução para Suporte Multiplataforma.
- **Status:** Aceito.
- **Contexto:** O editor precisa funcionar em Windows, Linux, macOS e também em ambiente web. Cada ambiente possui APIs diferentes para sistema de arquivos, processos, janelas, rede e armazenamento.

- **Decisão:** Separar implementações específicas de ambiente das abstrações compartilhadas. O código distingue contextos como common, browser, node e electron-main, preservando interfaces comuns sempre que possível.
- **Consequências:** A mesma base de código pode atender múltiplas plataformas, reduzindo duplicação. Em contrapartida, recursos dependentes de APIs nativas podem não estar disponíveis no navegador ou exigir implementações alternativas.
- **Evidências:** A estrutura do repositório separa módulos comuns e específicos de navegador, Node.js e Electron. O VS Code é distribuído para sistemas desktop e possui a versão web acessível em vscode.dev.

Característica: Interoperabilidade e Reutilização

- **Título:** ADR-006 — Uso de Protocolos Abertos para Linguagens e Depuração.
- **Status:** Aceito.
- **Contexto:** O VS Code precisa integrar ferramentas construídas em diferentes linguagens e ecossistemas, como compiladores, analisadores estáticos, depuradores e servidores de linguagem. Criar uma integração proprietária para cada ferramenta aumentaria o acoplamento e o custo de manutenção.
- **Decisão:** Adotar protocolos independentes de linguagem e editor, especialmente o **Language Server Protocol (LSP)** e o **Debug Adapter Protocol (DAP)**. As extensões atuam como adaptadores entre o VS Code e ferramentas externas.
- **Consequências:** Servidores de linguagem e adaptadores de depuração podem ser reutilizados por outros editores compatíveis. A desvantagem é que nem toda capacidade específica de uma ferramenta se encaixa perfeitamente no protocolo genérico.
- **Evidências:** As especificações do [Language Server Protocol](#) e do [Debug Adapter Protocol](#) são públicas e independentes da implementação interna do VS Code.

Característica: Segurança

- **Título:** ADR-007 — Execução Restrita de Workspaces Não Confiáveis.
- **Status:** Aceito.
- **Contexto:** Abrir uma pasta desconhecida pode acionar tarefas, configurações, extensões, depuradores ou scripts que executam código automaticamente. Isso expõe usuários a projetos potencialmente maliciosos.
- **Decisão:** Adotar o mecanismo **Workspace Trust**, que abre workspaces não confiáveis em **Restricted Mode** e limita funcionalidades capazes de executar código ou interagir diretamente com o sistema.
- **Consequências:** O usuário ganha uma camada adicional de proteção ao inspecionar projetos desconhecidos. Em contrapartida, alguns recursos do editor ficam indisponíveis até que o usuário marque a pasta como confiável.
- **Evidências:** A documentação de Workspace Trust informa que o Restricted Mode limita ou desabilita recursos como terminal, tarefas, depuração, configurações de workspace e extensões. O projeto também adotou sandboxing de processos para reduzir privilégios da interface.

Característica: Configurabilidade

- **Título:** ADR-008 — Configuração em Camadas para Adaptar o Editor ao Usuário e ao Projeto.
- **Status:** Aceito.
- **Contexto:** Desenvolvedores possuem preferências individuais, enquanto projetos precisam impor convenções compartilhadas de formatação, linguagem, ferramentas e comportamento do editor.
- **Decisão:** Organizar configurações em escopos com precedência, incluindo valores padrão, configurações de usuário, configurações de workspace e, em workspaces com múltiplas pastas, configurações por pasta. Complementar esse modelo com atalhos, temas e perfis de usuário.
- **Consequências:** O usuário pode personalizar sua experiência sem necessariamente alterar as regras do projeto. Ao mesmo tempo, configurações em múltiplas camadas podem gerar conflitos ou dificultar a identificação da origem de um valor efetivo.

- **Evidências:** A documentação de Settings descreve escopos de usuário e workspace, armazenados em JSON. O editor também oferece personalização de atalhos, temas e Profiles.

Característica: Modularidade

- **Título:** ADR-009 — Injeção de Dependências para Desacoplar Serviços do Núcleo.
- **Status:** Aceito.
- **Contexto:** O núcleo precisa coordenar serviços como arquivos, configuração, armazenamento, telemetria, notificações e comandos sem que cada componente conheça diretamente a implementação concreta dos demais.
- **Decisão:** Utilizar injeção de dependências por meio do serviço `IInstantiationService`. Os serviços são registrados com identificadores criados por `createDecorator`, e os consumidores dependem de contratos em vez de instanciar diretamente suas dependências.
- **Consequências:** Componentes se tornam mais substituíveis, reutilizáveis e fáceis de testar. Em contrapartida, o fluxo de criação dos objetos deixa de ser totalmente visível no ponto de uso, exigindo conhecimento da infraestrutura de registro.
- **Evidências:** O mecanismo está concentrado em `src/vs/platform/instantiation`, incluindo os contratos `IInstantiationService` e `createDecorator`.

Característica: Agilidade, Evolutibilidade e Implantabilidade

- **Título:** ADR-010 — Evolução Contínua com Compatibilidade Gradual da API de Extensões.
- **Status:** Aceito.
- **Contexto:** O editor precisa incorporar melhorias frequentes sem quebrar extensões de terceiros, que são parte essencial do ecossistema. Uma alteração incompatível em uma API pode afetar milhares de usuários e mantenedores.

- **Decisão:** Manter lançamentos regulares, disponibilizar uma versão **Insiders** para validação antecipada e introduzir recursos de extensão de forma gradual, inclusive por meio de **Proposed API** antes de sua estabilização.
- **Consequências:** Novidades podem ser testadas antes de chegar à versão estável, reduzindo o risco de quebra ampla. O custo é a necessidade de manter compatibilidade, documentar depreciações e sustentar versões transitórias da API.
- **Evidências:** O projeto possui ciclos regulares de lançamento, canal Insiders e documentação específica para Proposed API.

Característica: Testabilidade

- **Título:** ADR-011 — Automação de Testes no Núcleo e no Ecossistema de Extensões.
- **Status:** Aceito.
- **Contexto:** Alterações no núcleo ou em extensões podem afetar múltiplas plataformas, linguagens e fluxos de desenvolvimento. A validação manual isolada não é suficiente para garantir segurança em cada release.
- **Decisão:** Manter suítes automatizadas para o núcleo, incluindo testes unitários, de integração e smoke tests, e oferecer ferramentas para que autores de extensões executem testes contra uma instância controlada do VS Code.
- **Consequências:** Regressões podem ser identificadas antes da distribuição de uma versão. Em contrapartida, testes de integração e interface tendem a ser mais lentos, frágeis e dependentes do ambiente do que testes unitários.
- **Evidências:** O repositório possui diretórios e pipelines de testes. Para extensões, o projeto publica o pacote @vscode/test-electron, usado para baixar, iniciar e testar uma instância do VS Code.

Característica: Acessibilidade

- **Título:** ADR-012 — Acessibilidade como Capacidade Transversal do Workbench.
- **Status:** Aceito.

- **Contexto:** Um editor de código precisa atender pessoas com diferentes necessidades visuais, auditivas, motoras e cognitivas. Se a acessibilidade for tratada apenas como complemento visual, recursos centrais podem permanecer inacessíveis.
- **Decisão:** Incorporar recursos de acessibilidade ao workbench, incluindo compatibilidade com leitores de tela, navegação por teclado, sinais sonoros e temas de alto contraste.
- **Consequências:** O sistema atende um público mais amplo e oferece alternativas de interação para tarefas relevantes. A contrapartida é que novos componentes de interface precisam obedecer continuamente requisitos semânticos, de foco e de navegação.
- **Evidências:** A documentação oficial de Accessibility descreve suporte a leitores de tela, configurações de acessibilidade, sinais auditivos e personalizações visuais.

1.5 CARACTERÍSTICAS ARQUITETURAIS - TOP 4

Característica: Extensibilidade

- **Título:** ADR-001 — Extensibilidade por meio de Extensões e Contratos Públicos.
- **Status:** Aceito.
- **Contexto:** O VS Code precisa suportar diferentes linguagens, ferramentas, fluxos de depuração, temas e integrações sem que cada nova funcionalidade exija alteração no núcleo do editor.
- **Decisão:** Expor uma API pública de extensões e permitir que funcionalidades sejam adicionadas por extensões. Cada extensão declara suas capacidades, como comandos, linguagens, temas, depuradores e visualizações, no arquivo `package.json`.
- **Consequências:** Novas funcionalidades podem ser criadas e distribuídas sem modificar o núcleo do VS Code. Em contrapartida, a Microsoft precisa manter compatibilidade com a API pública para não quebrar extensões já existentes.

- **Evidências:** O diretório `extensions/` do repositório contém extensões embarcadas, como Git, TypeScript e Markdown. O contrato público está em `src/vscode-dts/vscode.d.ts`, e os *contribution points* definem como extensões registram suas funcionalidades.

Característica: Confiabilidade e Tolerância a Falhas

- **Título:** ADR-002 — Isolamento das Extensões em um Processo Separado.
- **Status:** Aceito.
- **Contexto:** O VS Code executa extensões criadas por milhares de autores. Uma extensão pode apresentar erro, consumir muitos recursos ou travar durante sua execução. Essas falhas não devem encerrar toda a aplicação nem interromper o trabalho do usuário.
- **Decisão:** Executar extensões em um processo separado chamado **Extension Host**. A interface principal e o núcleo do editor se comunicam com esse processo por mensagens, sem executar diretamente o código das extensões.
- **Consequências:** Uma falha no Extension Host pode ser tratada sem fechar toda a janela do editor. O sistema pode informar o problema e permitir o reinício do host. Em contrapartida, a comunicação entre núcleo e extensões tem custo maior, pois os dados precisam ser serializados e enviados entre processos.
- **Evidências:** O VS Code exibe notificações como *“Extension host terminated unexpectedly”* quando esse processo falha. O recurso **Extension Bisect** ajuda a identificar extensões defeituosas, e o código do projeto contém mecanismos para registrar e reiniciar o Extension Host.

Característica: Desempenho

- **Título:** ADR-003 — Ativação Sob Demanda para Preservar o Desempenho do Editor.
- **Status:** Aceito.

- **Contexto:** O VS Code precisa continuar responsivo durante a digitação, a navegação entre arquivos e a execução de comandos, mesmo quando o usuário possui muitas extensões instaladas.
- **Decisão:** Carregar extensões apenas quando forem necessárias. A ativação pode ocorrer, por exemplo, ao abrir um arquivo de uma linguagem específica, executar um comando, iniciar uma depuração ou abrir uma visualização fornecida pela extensão.
- **Consequências:** O tempo de inicialização e o uso inicial de memória são menores, pois extensões desnecessárias não são carregadas imediatamente. Em contrapartida, autores de extensões precisam declarar corretamente os eventos de ativação para evitar atrasos no momento em que o recurso for utilizado.
- **Evidências:** A documentação de *Activation Events* descreve eventos como `onLanguage`, `onCommand`, `onDebug` e `onView`. O comando **Developer: Startup Performance** permite analisar o impacto da inicialização do editor e das extensões.

Característica: Interoperabilidade e Reutilização

- **Título:** ADR-004 — Integração com Ferramentas Externas por Protocolos Abertos.
- **Status:** Aceito.
- **Contexto:** O VS Code precisa integrar ferramentas desenvolvidas em diferentes linguagens, como compiladores, analisadores de código, servidores de linguagem e depuradores. Criar uma integração própria para cada ferramenta aumentaria o acoplamento e dificultaria a manutenção.
- **Decisão:** Utilizar protocolos abertos, principalmente o **Language Server Protocol (LSP)** e o **Debug Adapter Protocol (DAP)**. As extensões atuam como adaptadores entre o VS Code e as ferramentas externas.
- **Consequências:** Um servidor de linguagem ou adaptador de depuração pode ser reutilizado por outros editores compatíveis com os mesmos protocolos. Em

contrapartida, recursos muito específicos de uma ferramenta podem exigir extensões ou APIs adicionais.

- **Evidências:** As especificações do LSP e do DAP são públicas e mantidas pela Microsoft. Extensões de linguagem e depuração do VS Code usam esses contratos para se comunicar com processos externos.

Justificativa da escolha do top 4

1. **Extensibilidade:** É a razão de ser do estilo Microkernel. Sem ela, o VS Code seria apenas um editor de texto com recursos fixos. A evidência mais forte é que a própria Microsoft entrega várias funcionalidades como extensões internas, usando o mesmo contrato público oferecido a terceiros. Essa característica fundamenta a API de extensões, os *contribution points* e o registro de contribuições no `package.json`.
2. **Confiabilidade e Tolerância a Falhas:** O sistema executa código criado por milhares de autores, sem garantia de qualidade. Por isso, foi necessário proteger o núcleo e a interface contra falhas das extensões. Essa característica justificou a decisão de executar plugins no Extension Host, um processo separado do editor principal. O custo é aceito: as chamadas entre o núcleo e as extensões precisam ser enviadas entre processos.
3. **Desempenho:** Um editor de código precisa transmitir sensação de rapidez durante a digitação, a abertura de arquivos e a execução de comandos. Uma extensão lenta não pode travar diretamente a interface. Essa característica motivou a ativação sob demanda das extensões e a comunicação assíncrona com o Extension Host. Dessa forma, o mesmo componente atende tanto ao desempenho quanto à tolerância a falhas.
4. **Interoperabilidade e Reutilização:** O VS Code precisa funcionar com ferramentas de diferentes linguagens e ecossistemas. Essa característica justificou a adoção do LSP e do DAP, que permitem conectar servidores de linguagem e adaptadores de depuração externos ao editor. Esses componentes também podem ser reutilizados por outras ferramentas compatíveis, sem depender exclusivamente do VS Code.

A Portabilidade e a Segurança ficaram fora do top 4, apesar de relevantes. A Portabilidade é atendida principalmente por decisões de tecnologia e ambiente de execução, como Electron, Node.js, navegador e abstrações específicas para Windows, Linux e macOS. Ela é importante para distribuir o produto, mas não é a principal motivadora da estrutura Microkernel.

A Segurança é tratada principalmente por mecanismos aplicados sobre a arquitetura já definida, como sandboxing, Workspace Trust e Restricted Mode. Esses recursos diminuem os riscos de abrir projetos ou executar extensões não confiáveis, mas não são o principal motivo para a separação entre núcleo e extensões.

1.6 COMPONENTES CANDIDATOS

Microkernel (Sistema Central do VS Code)

- **Responsabilidades:** Coordenar o funcionamento geral do editor, manter os contratos internos, iniciar os serviços principais e permitir que extensões adicionem recursos sem alterar diretamente o núcleo.
- **Requisitos do Sistema:** Atende à necessidade de oferecer um editor estável, capaz de evoluir com novas linguagens, ferramentas e integrações.
- **Características Arquiteturais:** É a base da Extensibilidade, pois fornece os pontos de integração usados pelos plugins. Também apoia a Confiabilidade, ao manter o funcionamento principal separado do código das extensões.

Processo Principal (Electron Main)

- **Responsabilidades:** Controlar o ciclo de vida da aplicação, criar janelas, integrar o editor ao sistema operacional e coordenar funções do ambiente desktop.
- **Requisitos do Sistema:** Atende à necessidade de executar o VS Code como aplicação nativa em Windows, Linux e macOS.
- **Características Arquiteturais:** Representa a Portabilidade, pois concentra a comunicação com recursos específicos de cada sistema operacional. **Também** apoia a Agilidade, permitindo atualização e controle central da aplicação desktop.

Interface do Usuário (Workbench)

- **Responsabilidades:** Montar a interface principal do VS Code, incluindo barra de atividades, painéis, abas, menus, paleta de comandos e áreas onde extensões inserem visualizações.
- **Requisitos do Sistema:** Atende à necessidade de oferecer uma interface organizada para edição de código, navegação de arquivos, execução de comandos e uso de extensões.
- **Características Arquiteturais:** Representa a Extensibilidade, pois permite que plugins adicionem menus, comandos e *views*. Também apoia a Acessibilidade, por concentrar navegação por teclado, foco, leitores de tela e temas de alto contraste.

Editor de Texto (Monaco Editor)

- **Responsabilidades:** Gerenciar o modelo de texto, renderizar o código, controlar cursores, seleção, desfazer, refazer e os pontos de integração para recursos de linguagem.
- **Requisitos do Sistema:** Cumpre a necessidade principal de editar arquivos de código de forma rápida, precisa e responsiva.
- **Características Arquiteturais:** É direcionado ao Desempenho, pois precisa responder imediatamente à digitação e à navegação, mesmo em arquivos grandes.

Serviços de Plataforma e Injeção de Dependências (src/vs/platform)

- **Responsabilidades:** Fornecer serviços compartilhados, como logs, armazenamento, configuração, telemetria e ciclo de vida de componentes. Também cria e conecta serviços por meio do `IInstantiationService`.
- **Requisitos do Sistema:** Resolve a necessidade de reutilizar serviços internos sem que cada componente conheça diretamente a implementação dos demais.

- **Características Arquiteturais:** Favorece a Modularidade e a Testabilidade, pois componentes dependem de contratos e podem usar implementações substituíveis durante testes.

Serviço de Arquivos (IFileService)

- **Responsabilidades:** Oferecer acesso padronizado a arquivos e pastas por meio de provedores de sistema de arquivos, como disco local, máquina remota ou sistema virtual.
- **Requisitos do Sistema:** Atende à necessidade de abrir, salvar e manipular arquivos em ambientes locais, remotos e web sem alterar o restante do editor.
- **Características Arquiteturais:** Representa a Portabilidade, pois abstrai diferenças entre ambientes. Também apoia a Extensibilidade, porque novos provedores de arquivos podem ser integrados ao sistema.

Configuração e Perfis

- **Responsabilidades:** Resolver configurações em camadas, como padrão, usuário, workspace e pasta, além de armazenar e aplicar perfis de usuário.
- **Requisitos do Sistema:** Atende à necessidade de permitir que cada usuário personalize o editor sem substituir configurações definidas por um projeto.
- **Características Arquiteturais:** Representa a Configurabilidade, pois adapta o comportamento do VS Code às preferências do usuário e às regras do workspace.

Registro de Comandos e Atalhos (CommandsRegistry)

- **Responsabilidades:** Registrar comandos do editor e das extensões, conectando menus, paleta de comandos, atalhos de teclado e ações da interface.
- **Requisitos do Sistema:** Resolve a necessidade de disponibilizar uma forma única de acionar funcionalidades, independentemente de serem nativas ou fornecidas por extensões.

- **Características Arquiteturais:** Impulsiona a Extensibilidade, pois extensões podem registrar novos comandos. Também atende à Configurabilidade, porque o usuário pode personalizar atalhos.

Workspace Trust

- **Responsabilidades:** Avaliar se um workspace é confiável e restringir recursos que possam executar código quando o projeto for aberto em modo restrito.
- **Requisitos do Sistema:** Atende à necessidade de reduzir riscos ao abrir projetos de origem desconhecida.
- **Características Arquiteturais:** Representa a Segurança, pois limita recursos como tarefas, terminal, depuração e extensões em workspaces não confiáveis.

Terminal Integrado (Pty Host)

- **Responsabilidades:** Executar e controlar os processos dos terminais integrados em um processo próprio, separado da interface principal do editor.
- **Requisitos do Sistema:** Atende à necessidade de executar comandos de desenvolvimento sem bloquear diretamente a interface do VS Code.
- **Características Arquiteturais:** Apoia o Desempenho e a Tolerância a Falhas, pois a execução do terminal ocorre fora do processo da interface.

Registro de Contribuições

- **Responsabilidades:** Ler os manifestos das extensões, validar suas declarações e registrar comandos, linguagens, menus, temas, configurações e outras contribuições.
- **Requisitos do Sistema:** Cumprir a necessidade de reconhecer de forma padronizada os recursos oferecidos por extensões.
- **Características Arquiteturais:** Representa diretamente a Extensibilidade, pois funciona como o mecanismo de registro de plugins do VS Code.

Serviço de Extensões (Extension Service)

- **Responsabilidades:** Descobrir extensões instaladas, decidir onde cada uma será executada — localmente, no navegador ou em ambiente remoto — disparar eventos de ativação e acompanhar o estado dos Extension Hosts.
- **Requisitos do Sistema:** Atende à necessidade de carregar extensões de forma controlada e somente quando forem necessárias.
- **Características Arquiteturais:** Atende ao Desempenho, por ativar extensões sob demanda. Também apoia a Confiabilidade e a Portabilidade, pois acompanha hosts locais, remotos e web.

Extension Host

- **Responsabilidades:** Executar o código das extensões em processo separado e trocar mensagens com o núcleo do editor.
- **Requisitos do Sistema:** Resolve a necessidade de executar plugins de terceiros sem permitir que uma falha deles afete diretamente a interface principal.
- **Características Arquiteturais:** É essencial para a Confiabilidade e Tolerância a Falhas, conforme o ADR-002. Também apoia o Desempenho, pois o trabalho das extensões não ocorre diretamente no processo da interface, e reforça a Segurança por separar contextos de execução.

Interface de Plugins e Protocolo RPC (`vscode.d.ts` e `extHost.protocol.ts`)

- **Responsabilidades:** Definir a API pública usada por extensões e o protocolo de comunicação entre o núcleo e o Extension Host.
- **Requisitos do Sistema:** Cumpre a necessidade de oferecer um contrato estável para que extensões sejam criadas por terceiros e funcionem sem acessar diretamente os componentes internos do editor.
- **Características Arquiteturais:** Representa a Extensibilidade, pois permite integrar novos plugins sem alterar o núcleo. Também favorece a Agilidade, porque novas capacidades podem evoluir de forma independente.

Gerenciamento de Extensões (`src/vs/platform/extensionManagement`)

- **Responsabilidades:** Instalar, atualizar, desinstalar, habilitar e desabilitar extensões, além de integrar o editor ao Marketplace.
- **Requisitos do Sistema:** Atende à necessidade de permitir que usuários adicionem ou removam funcionalidades de forma simples.
- **Características Arquiteturais:** Apoia a Agilidade, pois extensões podem ser atualizadas independentemente do editor. Também contribui para a Segurança, ao centralizar o gerenciamento de extensões instaladas.

Servidor Remoto (src/vs/server)

- **Responsabilidades:** Executar serviços do VS Code em outra máquina, WSL ou container, incluindo Extension Host, acesso a arquivos e terminal.
- **Requisitos do Sistema:** Resolve a necessidade de desenvolver em ambientes remotos sem perder a experiência da interface local do editor.
- **Características Arquiteturais:** Representa a Portabilidade, pois permite executar partes do sistema em ambientes diferentes. Também atende à Interoperabilidade, por conectar o editor local a ferramentas e recursos externos.

Extensões Embarcadas (extensions/)

- **Responsabilidades:** Entregar funcionalidades padrão do VS Code, como integração com Git, recursos para TypeScript, Markdown, temas e suporte a linguagens.
- **Requisitos do Sistema:** Atendem à necessidade de fornecer recursos importantes já na instalação inicial, sem tornar essas funcionalidades parte fixa do núcleo.
- **Características Arquiteturais:** Demonstram a Extensibilidade, pois usam o mesmo modelo de extensões disponível para desenvolvedores externos.

Extensões de Terceiros

- **Responsabilidades:** Adicionar linguagens, ferramentas, integrações, temas, depuradores e funcionalidades distribuídas pelo Marketplace.

- **Requisitos do Sistema:** Atendem à necessidade de adaptar o VS Code a diferentes equipes, tecnologias e fluxos de trabalho.
- **Características Arquiteturais:** São a principal expressão da Extensibilidade, permitindo que o ecossistema evolua sem exigir alterações no núcleo do editor.

Servidores de Linguagem (LSP)

- **Responsabilidades:** Executar análise de código para linguagens específicas, oferecendo recursos como autocompletar, diagnósticos, navegação, *hover* e renomeação de símbolos.
- **Requisitos do Sistema:** Resolvem a necessidade de oferecer suporte avançado a várias linguagens sem implementar toda a lógica dentro do VS Code.
- **Características Arquiteturais:** Representam a Interoperabilidade e Reutilização, conforme o ADR-004, pois se comunicam pelo Language Server Protocol e podem ser usados por outros editores compatíveis.

Adaptadores de Depuração (DAP)

- **Responsabilidades:** Conectar o VS Code a depuradores específicos de cada linguagem ou ambiente de execução.
- **Requisitos do Sistema:** Atendem à necessidade de iniciar programas, definir *breakpoints*, inspecionar variáveis e controlar a execução de código em diferentes tecnologias.
- **Características Arquiteturais:** Representam a Interoperabilidade, pois usam o Debug Adapter Protocol para padronizar a comunicação entre o editor e depuradores externos.

1.7 DIAGRAMA DE COMPONENTES DA ARQUITETURA

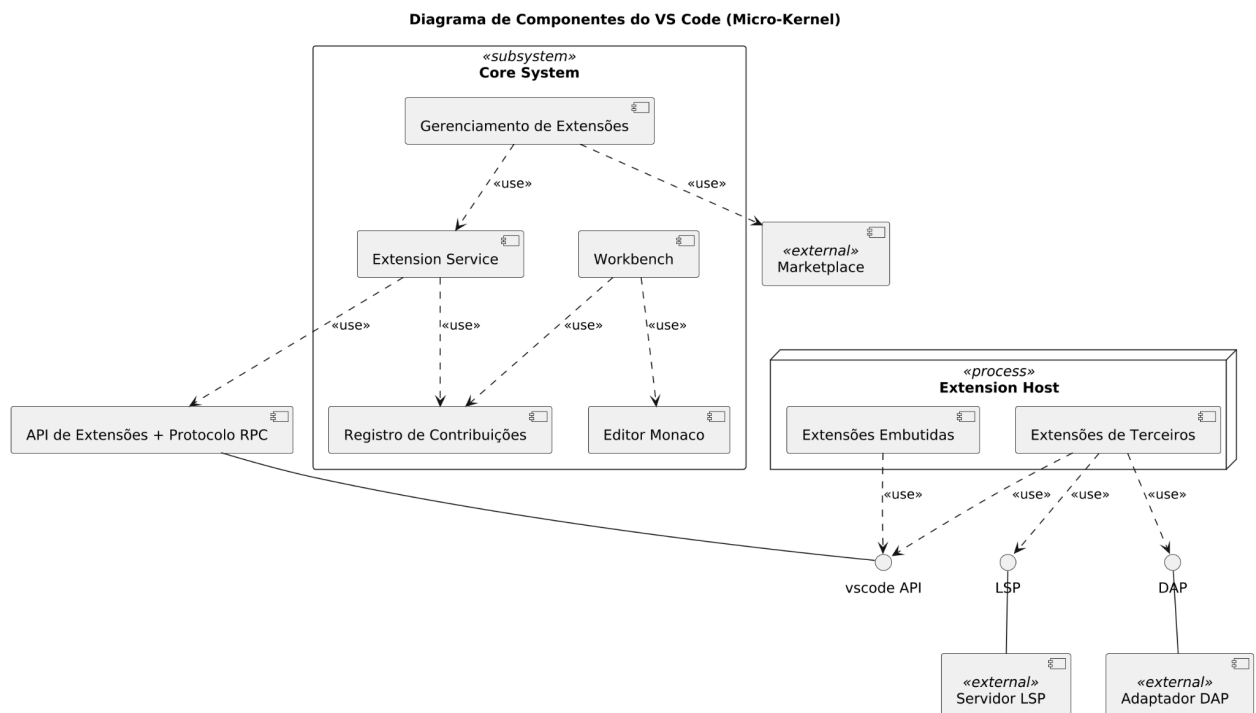


Diagrama de Componentes do VS Code

Fonte: Elaboração própria (2026).

2. OBS STUDIO

Repositório do GitHub: [obsproject/obs-studio](https://github.com/obsproject/obs-studio): [OBS Studio - Free and open source software for live streaming and screen recording](https://obsproject.com/)

2.1 CARACTERÍSTICAS ARQUITETURAIS

Característica: Agilidade

- **Título:** ADR-001 Adoção do Estilo Microkernel para Favorecer a Agilidade.
- **Status:** Aceito.
- **Contexto:** O sistema exige suporte contínuo a novos hardwares e plataformas de *streaming*, precisando responder rapidamente a mudanças e ser mantido por centenas de colaboradores *open source* sem degradar o núcleo principal.

- **Decisão:** Foi definida a priorização da agilidade por meio da adoção do estilo Microkernel, que estabelece o isolamento de responsabilidades e a criação de contratos de API claros.
- **Consequências:** A comunidade de desenvolvedores e os fabricantes podem criar módulos de forma independente. O código se mantém legível e modular, e novos *plugins* não quebram o motor central.
- **Evidências:** O diretório *plugins/* do repositório abriga as funcionalidades periféricas, e a estrutura de diretórios separa a interface (diretório *frontend/*) do núcleo (*libobs*), que oferece uma API em C (*obs-module.h*).

Característica: Desempenho

- **Título:** ADR-002 Priorização do Desempenho no Processamento Multimídia.
- **Status:** Aceito.
- **Contexto:** Atrasos ou bloqueios nos processos de vídeo resultam em perda de quadros durante o processamento em tempo real.
- **Decisão:** Garantir comunicação de baixa latência entre o sistema central e os *plugins* de codificação, minimizando a sobrecarga causada pela cópia de memória.
- **Consequências:** Transmissão ao vivo fluida. O sistema consegue sustentar altas taxas de atualização em tempo real sem esgotar o processador do computador.
- **Evidências:** O núcleo (*libobs*) foi desenvolvido predominantemente em linguagem C para maximizar a velocidade. Observa-se também o compartilhamento de texturas diretamente na memória da GPU.

Característica: Confiabilidade e Tolerância a Falhas

- **Título:** ADR-003 Priorização da Confiabilidade e da Tolerância a Falhas em Ambientes de *Broadcasting*.
- **Status:** Aceito.
- **Contexto:** Durante uma transmissão ao vivo, interrupções ou falhas no *pipeline* central de áudio e vídeo causam grande impacto para o usuário. Além disso, erros de hardware ou falhas de rede não podem causar o encerramento ou o bloqueio de toda a aplicação durante uma gravação.

- **Decisão:** Isolar estritamente o *loop* principal de renderização multimídia, de modo que o núcleo não dependa do tempo de resposta de serviços externos, e executar de forma assíncrona os módulos propensos a falhas externas, com tratamento local de exceções.
- **Consequências:** Garante a continuidade do serviço básico, sem congelar nem corromper a captura principal, mesmo sob uso intenso. Se a rede cair, o *plugin* de saída tenta se reconectar de forma autônoma, sem bloquear a renderização nem congelar a interface gráfica.
- **Evidências:** A implementação do *libobs*, que gerencia a orquestração e a sincronização de *threads* de forma independente dos *plugins* periféricos, e as *threads* assíncronas distintas para a comunicação de rede e a captura de dispositivos, separadas do *loop* principal.

Característica: Capacidade de Aprendizado

- **Título:** ADR-004 Foco na Capacidade de Aprendizado por meio de uma Interface Gráfica Rica.
- **Status:** Aceito.
- **Contexto:** O processamento de áudio e vídeo em tempo real é complexo, mas o usuário final precisa de ferramentas visuais, rápidas e intuitivas para aprender a operar o sistema sem dificuldades.
- **Decisão:** Construir um *frontend* interativo e dedicado, capaz de fornecer *feedback* visual em tempo real e mantido como uma camada totalmente separada do motor de processamento.
- **Consequências:** Reduz a curva de aprendizado para usuários iniciantes. Permite operações visuais complexas por meio de ações simples, evitando que a operação do sistema sobrecarregue o usuário final.
- **Evidências:** A implementação dos componentes na *framework* Qt, que oferece painéis modulares, o "Modo Estúdio" para transições seguras e a manipulação de fontes por arrastar e soltar.

Característica: Implantabilidade e Portabilidade

- **Título:** ADR-005 Maximização da Implantabilidade Multiplataforma.

- **Status:** Aceito.
- **Contexto:** O software precisa atender usuários de diferentes sistemas operacionais e ser distribuído de forma consistente em Windows, macOS e Linux.
- **Decisão:** Isolar e abstrair as chamadas nativas de cada sistema operacional (gerenciamento de janelas, E/S e renderização gráfica) e utilizar o carregamento dinâmico de bibliotecas.
- **Consequências:** Facilita a implantação, permitindo que o sistema seja empacotado em instaladores independentes e que novas funcionalidades específicas de cada sistema operacional sejam adicionadas sem poluir o núcleo.
- **Evidências:** A separação física dos pacotes no código-fonte (*win-capture*, *mac-capture* e *linux-capture*) e a adoção da *framework* Qt para uma UI unificada.

Característica: Extensibilidade

- **Título:** ADR-006 Extensibilidade por meio do Registro Padronizado de Módulos.
- **Status:** Aceito.
- **Contexto:** O ecossistema de captura, codificação e transmissão muda continuamente, com novos dispositivos, codecs e plataformas de *streaming*. A equipe central não consegue suportar tudo, e a comunidade e os fabricantes precisam adicionar funcionalidades sem alterar o núcleo.
- **Decisão:** Expor no *libobs* uma API de registro de módulos, na qual cada *plugin* declara, ao ser carregado, as fontes, os filtros, os codificadores e as saídas que oferece. O núcleo apenas consome essas capacidades por meio de contratos padronizados.
- **Consequências:** Novos tipos de fonte, codificador e saída podem ser adicionados sem modificar o núcleo, e a maior parte das funcionalidades visíveis ao usuário é entregue como módulos. Em contrapartida, o núcleo precisa manter a compatibilidade da API, e os módulos são executados no mesmo processo dele.
- **Evidências:** O diretório *plugins/* reúne os módulos que acompanham o OBS (como *obs-x264*, *obs-ffmpeg* e *obs-outputs*) e usa o mesmo contrato oferecido a

terceiros: o cabeçalho *obs-module.h*, a função *obs_module_load()* e as funções de registro *obs_register_source()*, *obs_register_encoder()* e *obs_register_output()*.

Característica: Testabilidade

- **Título:** ADR-007 Separação de Módulos para Alta Testabilidade.
- **Status:** Aceito.
- **Contexto:** No contexto do OBS, é essencial garantir que novas funcionalidades e refatorações possam ser validadas automaticamente, para não quebrarem a lógica existente antes de cada lançamento.
- **Decisão:** Manter forte desacoplamento entre o núcleo (*libobs*), a interface gráfica e os *plugins* externos.
- **Consequências:** Promove alta testabilidade, permitindo que a lógica de negócio central e o processamento de quadros sejam testados de forma independente do estado da interface gráfica ou do hardware.
- **Evidências:** O isolamento garantido pelos cabeçalhos da API em C, que permite invocar e simular funcionalidades do núcleo sem inicializar o *frontend* Qt.

2.2 CARACTERÍSTICAS ARQUITETURAIS - TOP 4

Característica: Extensibilidade

- **Título:** ADR-006 Extensibilidade por meio do Registro Padronizado de Módulos.
- **Status:** Aceito.
- **Contexto:** A comunidade e os fabricantes precisam adicionar novos dispositivos, codecs e plataformas sem alterar o núcleo.
- **Decisão:** Expor no *libobs* uma API de registro de módulos, na qual cada *plugin* declara, ao ser carregado, as fontes, os filtros, os codificadores e as saídas que oferece.
- **Consequências:** Novos tipos de fonte, codificador e saída podem ser adicionados sem modificar o núcleo. Em contrapartida, o núcleo precisa manter a compatibilidade da API.

- **Evidências:** O diretório *plugins/* e o contrato *obs-module.h*, com a função *obs_module_load()* e as funções de registro de fontes, codificadores e saídas.

Característica: Desempenho

- **Título:** ADR-002 Priorização do Desempenho no Processamento Multimídia.
- **Status:** Aceito.
- **Contexto:** Atrasos ou bloqueios nos processos de vídeo resultam em perda de quadros durante o processamento em tempo real.
- **Decisão:** Garantir comunicação de baixa latência entre o sistema central e os *plugins* de codificação, minimizando a sobrecarga causada pela cópia de memória.
- **Consequências:** Transmissão ao vivo fluida. O sistema consegue sustentar altas taxas de atualização em tempo real.
- **Evidências:** O núcleo (*libobs*) foi desenvolvido predominantemente em linguagem C para maximizar a velocidade.

Característica: Confiabilidade e Tolerância a Falhas

- **Título:** ADR-003 Priorização da Confiabilidade e da Tolerância a Falhas em Ambientes de *Broadcasting*.
- **Status:** Aceito.
- **Contexto:** Durante uma transmissão ao vivo, interrupções ou falhas no *pipeline* central de áudio e vídeo causam grande impacto para o usuário. Além disso, erros de hardware ou falhas de rede não podem encerrar nem bloquear a aplicação durante uma gravação.
- **Decisão:** Isolar estritamente o *loop* principal de renderização, de modo que o núcleo não dependa do tempo de resposta de serviços externos, e executar de forma assíncrona os módulos propensos a falhas externas, com tratamento local de exceções.
- **Consequências:** Continuidade do serviço básico, sem congelar nem corromper a captura, mesmo sob uso intenso. Em caso de queda da rede, o *plugin* de saída tenta se reconectar de forma autônoma.

- **Evidências:** A implementação do *libobs*, que gerencia a orquestração e a sincronização de *threads* de forma independente dos *plugins* periféricos, e as *threads* assíncronas para rede e captura.

Característica: Implantabilidade e Portabilidade

- **Título:** ADR-005 Maximização da Implantabilidade Multiplataforma.
- **Status:** Aceito.
- **Contexto:** O software precisa atender usuários de diferentes sistemas operacionais e ser distribuído de forma consistente em Windows, macOS e Linux.
- **Decisão:** Isolar e abstrair as chamadas nativas de cada sistema operacional e utilizar o carregamento dinâmico de bibliotecas.
- **Consequências:** Facilita a implantação, permitindo que o sistema seja empacotado em instaladores independentes.
- **Evidências:** A separação física dos pacotes no código-fonte (*win-capture*, *mac-capture* e *linux-capture*) e a adoção da *framework* Qt para uma UI unificada.

Justificativa da escolha do top 4

1. Extensibilidade: É a razão de ser do estilo Microkernel. Sem ela, o OBS seria apenas um compositor de cenas com um conjunto fixo de fontes e codificadores. A evidência mais forte é que as próprias funcionalidades padrão, como captura, codificação e saída, são entregues como módulos em *plugins/*, pelo mesmo contrato (*obs-module.h*) oferecido a terceiros. Essa característica fundamenta a Plugin Interface e os Plugins de Fontes, de Codificação e de Saída.

2. Desempenho: Transmitir e gravar em tempo real a 30 ou 60 quadros por segundo não admite atrasos, e um quadro perdido é percebido de imediato. Essa característica motivou a implementação do núcleo em C, o compartilhamento de texturas na GPU sem cópia (ADR-IMP-003) e a comunicação de baixa latência com os *plugins* de codificação. É também a que mais restringe os *plugins*, que precisam executar no contexto gráfico controlado pelo núcleo.

3. Confiabilidade e Tolerância a Falhas: Uma transmissão ao vivo não pode ser repetida: se o programa travar ou a gravação for corrompida, o conteúdo se perde. Como o sistema depende de redes instáveis e de dispositivos de terceiros, a falha externa é esperada, e não excepcional. Isso justificou isolar o *loop* de renderização e executar rede e captura em *threads* próprias, com reconexão automática. Como os módulos são carregados no mesmo processo do núcleo, a proteção vem do tratamento local de erros e da separação de *threads*, e não do isolamento de processos, como no VS Code.

4. Implantabilidade e Portabilidade: O OBS é distribuído para Windows, macOS e Linux e precisa receber novos módulos sem recompilar o núcleo. Essa característica justificou a abstração das chamadas nativas do sistema operacional, a separação dos módulos de captura por plataforma (*win-capture*, *mac-capture* e *linux-capture*) e o carregamento dinâmico de bibliotecas (ADR-IMP-004).

A Agilidade, a Testabilidade e a Capacidade de Aprendizado ficaram fora do top 4, apesar de relevantes. A Agilidade é, em grande parte, consequência da Extensibilidade: como os módulos evoluem de forma independente, a comunidade entrega mudanças sem alterar o núcleo. A Testabilidade decorre do desacoplamento que o próprio estilo já exige, sendo um benefício da estrutura mais do que sua motivação. A Capacidade de Aprendizado é resolvida na camada de interface (Qt), que pode ser substituída sem afetar o motor, e por isso não determina a estrutura de componentes do sistema.

2.3 LISTA DE COMPONENTES CANDIDATOS

- **Microkernel (*libobs*)**

- **Responsabilidades:** Orquestrar o *pipeline* de áudio e vídeo, gerenciar o ciclo de vida da aplicação e carregar e descarregar os *plugins* em tempo de execução.
- **Requisitos do Sistema:** Satisfaz a necessidade de um gerenciador centralizado que sincronize com precisão as faixas de áudio e vídeo capturadas antes de enviá-las para codificação.

- **Características Arquiteturais:** Garante o Desempenho, por ser otimizado em C para latência mínima, e atua como pilar da Confiabilidade, protegendo a orquestração central de falhas externas.
- **Plugin Interface (*obs-module*)**
 - **Responsabilidades:** Definir as assinaturas de funções e os cabeçalhos que determinam como um *plugin* se registra, informa suas capacidades e se comunica com o *libobs*.
 - **Requisitos do Sistema:** Cumpre o requisito de padronizar e validar a comunicação segura entre código de terceiros e o motor principal.
 - **Características Arquiteturais:** Representa a Extensibilidade, permitindo o acoplamento de novos módulos sem alterar o código-fonte do núcleo.
- **Interface do Usuário (UI - Qt Frontend)**
 - **Responsabilidades:** Fornecer a interface gráfica (GUI) que permite ao usuário gerenciar cenas e fontes e iniciar transmissões, funcionando como um cliente que envia comandos ao *libobs*.
 - **Requisitos do Sistema:** Atende à necessidade de fornecer aos usuários finais um painel de controle visual, intuitivo e reativo.
 - **Características Arquiteturais:** Representa a Portabilidade, pois o uso da *framework* Qt permite compilar a mesma interface nativamente para Windows, macOS e Linux.
- **Plugins de Fontes (Source Plugins)**
 - **Responsabilidades:** Capturar dados brutos de dispositivos locais ou do sistema operacional.
 - **Requisitos do Sistema:** Resolve a necessidade de suportar uma ampla gama de dispositivos e métodos de entrada de imagem e som.
 - **Características Arquiteturais:** Impulsiona a Extensibilidade e respeita a Portabilidade por meio da separação em submódulos, como *win-capture* ou *linux-capture*.

- **Plugins de Codificação (Encoder Plugins)**

- **Responsabilidades:** Receber os *frames* compostos pelo núcleo e comprimi-los em formatos de vídeo e áudio padronizados.
- **Requisitos do Sistema:** Atende à necessidade de reduzir o tamanho dos arquivos de vídeo brutos para que possam ser transmitidos com eficiência pela internet.
- **Características Arquiteturais:** Direcionado ao Desempenho, integra-se às APIs da GPU para realizar a compressão sem sobrecarregar o processador central.

- **Plugins de Saída (Output Plugins)**

- **Responsabilidades:** Empacotar e enviar os dados codificados ao destino final, seja gravando em um arquivo local ou transmitindo via protocolo RTMP.
- **Requisitos do Sistema:** Cumpre o requisito principal do software: publicar a transmissão em plataformas de *streaming* externas ou arquivá-la localmente.
- **Características Arquiteturais:** Essencial para a Confiabilidade, pois opera em *threads* assíncronas dedicadas à rede. Em caso de perda de pacotes, gerencia as tentativas de reconexão sem congelar o restante do software.

2.4 DIAGRAMA DE COMPONENTES DA ARQUITETURA

O diagrama ilustra os componentes mais críticos do estilo Microkernel. O *libobs* e a API *obs-module* foram selecionados por serem o eixo central da arquitetura. Dentre o vasto ecossistema de extensões, foram documentados os *Source Plugins*, *Encoder Plugins* e *Output Plugins*, pois representam coletivamente o *pipeline* mínimo e essencial de funcionamento do software: capturar a imagem, comprimir os dados e enviá-los para a rede.

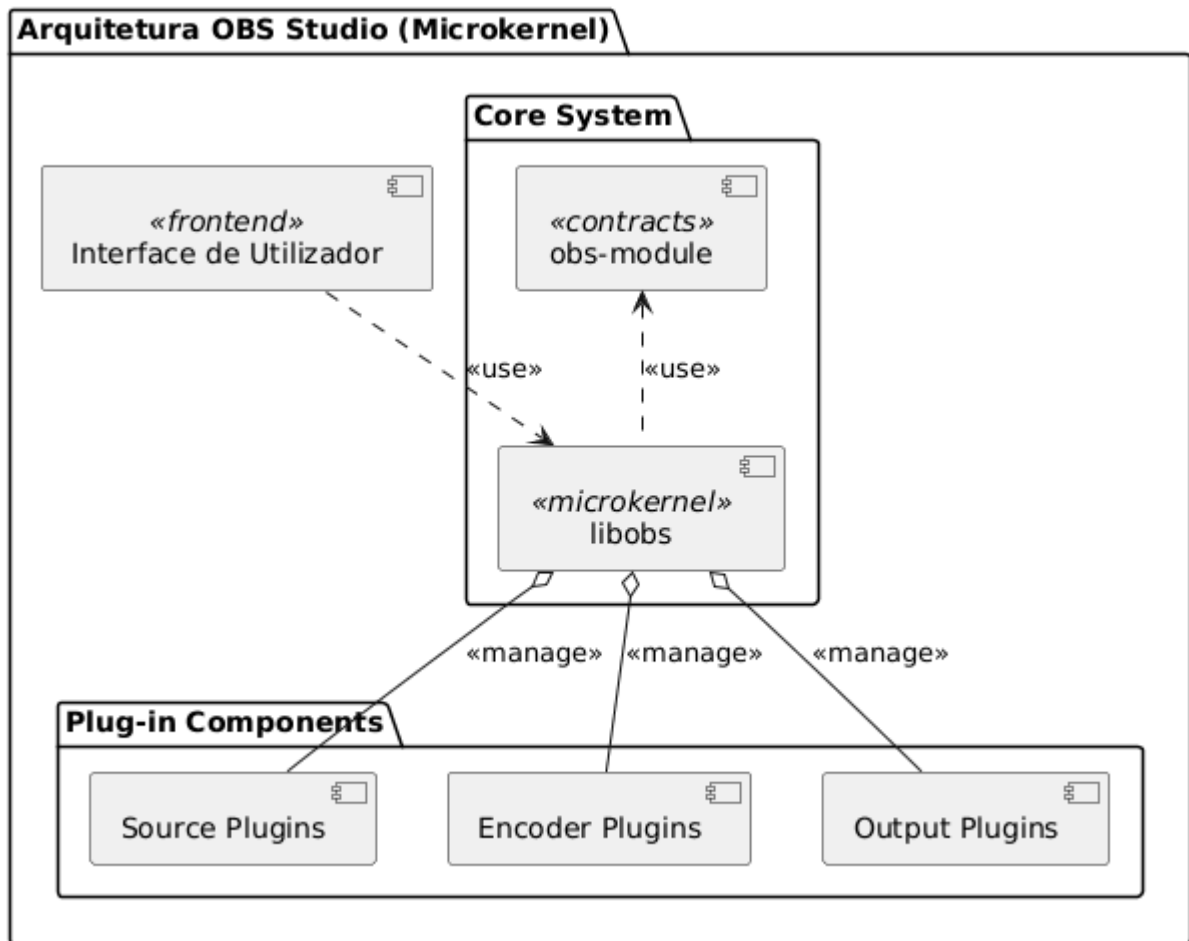


Diagrama de Componentes do OBS Studio

Fonte: Elaboração própria (2026).

2.5 DECISÕES ARQUITETURAIS

Decisão Arquitetural 1: Implementação do Núcleo (Microkernel) em Linguagem C

- **Título:** ADR-IMP-001 Adoção da Linguagem C para o Núcleo (*libobs*).
- **Status:** Aceito.
- **Contexto:** O motor de coordenação de áudio e vídeo precisa de controle sobre a gestão de memória, as *threads* e os ciclos de relógio do processador, além de exportar uma Interface Binária de Aplicação (ABI) estável, para que *plugins* desenvolvidos em outras linguagens possam se ligar a ele.

- **Decisão:** Foi decidido desenvolver o núcleo *libobs* em linguagem C, em vez de C++ ou outra linguagem de mais alto nível.
- **Consequências:** Favorece alto desempenho e baixa latência. A ABI do C facilita o desenvolvimento de *plugins* e de interfaces de usuário, mantendo o sistema central agnóstico em relação à tecnologia periférica.
- **Evidências:** Ao analisar o repositório, vê-se que o diretório *libobs/* é composto predominantemente por arquivos *.c* e cabeçalhos *.h*, evitando classes e bibliotecas padrão do C++ no núcleo do sistema.

Decisão Arquitetural 2: Separação Física da Interface do Usuário

- **Título:** ADR-IMP-002 Desacoplamento da Interface Visual por meio da *Framework Qt*.
- **Status:** Aceito.
- **Contexto:** Em um software multiplataforma, o código de janelas e controles visuais tende a ficar intrinsecamente ligado à lógica de negócio se não houver cuidado. Além disso, empresas terceiras podem querer usar o motor do OBS com interfaces próprias.
- **Decisão:** Foi decidido isolar completamente o desenvolvimento da interface do usuário, adotando C++ e a *framework Qt* para a UI, que se comunica com o núcleo apenas por meio de chamadas à API exportada (*obs.h*).
- **Consequências:** Permite a construção de clientes alternativos. A UI original em Qt pode ser atualizada ou substituída sem alterar uma única linha do código de processamento de vídeo.
- **Evidências:** O código-fonte apresenta um diretório *frontend/* distinto, escrito em C++ com arquivos *.ui* do Qt, que faz chamadas à API do *libobs*, sem lógica interna de orquestração multimídia.

Decisão Arquitetural 3: Passagem de Texturas em GPU (Zero-Copy) entre Módulos

- **Título:** ADR-IMP-003 Compartilhamento de Contexto Gráfico para Processamento de Vídeo.
- **Status:** Aceito.

- **Contexto:** Copiar os *frames* de vídeo da memória RAM para a placa gráfica e vice-versa, a cada filtro ou transformação aplicada, reduziria o desempenho do sistema e dificultaria a transmissão a 60 FPS.
- **Decisão:** Foi decidido implementar um sistema de contexto compartilhado de renderização. Os *plugins* passam entre si apenas as referências das texturas armazenadas na memória da GPU, evitando a cópia dos pixels.
- **Consequências:** Redução do uso de CPU e de RAM do sistema. Em contrapartida, limita os desenvolvedores de *plugins* visuais, que precisam executar seu código no contexto das *threads* gráficas controladas pelo OBS.
- **Evidências:** A existência e o uso intensivo da estrutura *gs_texture_t* e das funções da API gráfica (*Graphics Subsystem*) nos cabeçalhos C do *libobs*, que obrigam os módulos a se sincronizar com a GPU.

Decisão Arquitetural 4: Carregamento Dinâmico de Módulos

- **Título:** ADR-IMP-004 Injeção de Funcionalidades via Carregamento Dinâmico do Sistema Operacional.
- **Status:** Aceito.
- **Contexto:** A arquitetura Microkernel exige que o sistema central carregue os *plugins* sem precisar ser recompilado sempre que um usuário instala um novo módulo de terceiros.
- **Decisão:** Foi decidido utilizar as APIs nativas de carregamento dinâmico de bibliotecas de cada sistema operacional para ler arquivos binários presentes em uma pasta específica durante a inicialização.
- **Consequências:** Permite uma arquitetura verdadeiramente *plug and play*. O núcleo inicia, varre as pastas de *plugins* (como *obs-plugins/*), carrega as bibliotecas em tempo de execução e registra os *callbacks* (funções de retorno) de cada *plugin*.
- **Evidências:** A implementação da lógica de carregamento multiplataforma no arquivo *libobs/obs-module.c*, que define como os módulos externos são carregados na memória e iniciados pela função padronizada *obs_module_load()*, implementada por cada módulo.

2.6 DIAGRAMA DE MÓDULOS DA ARQUITETURA

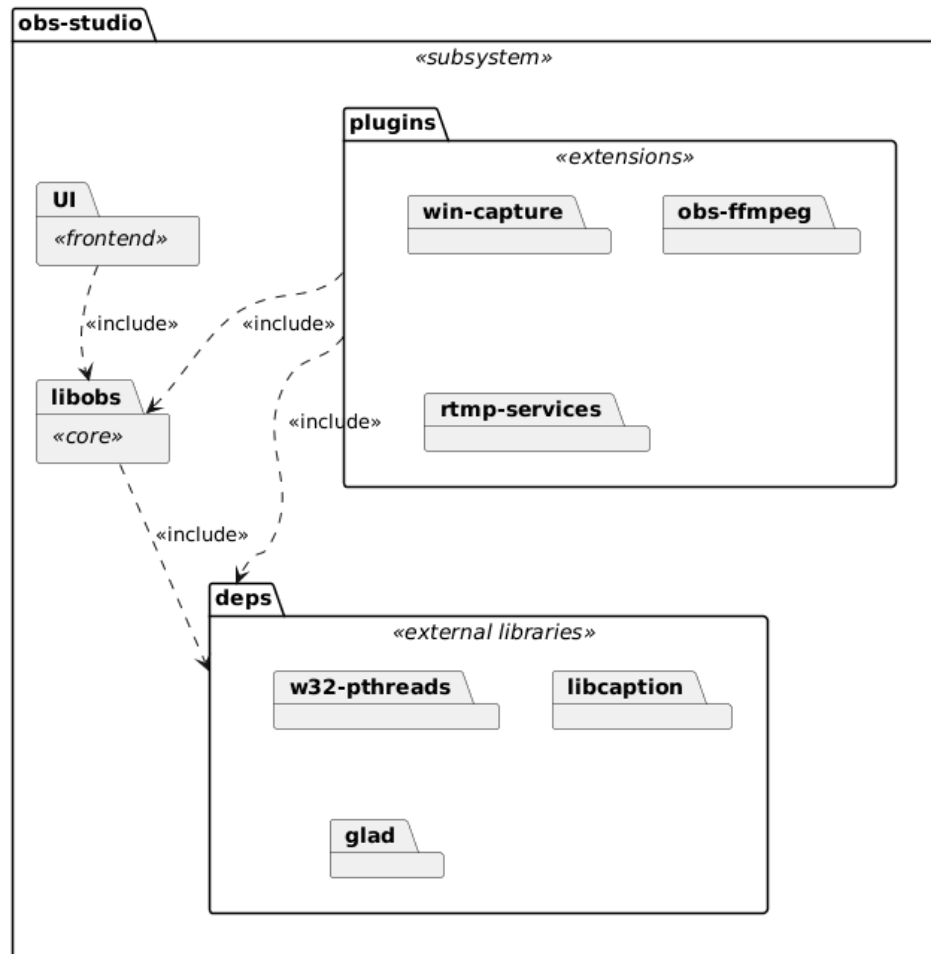


Diagrama de Módulos do OBS Studio

Fonte: Elaboração própria (2026).

2.7 TÁTICAS PARA CARACTERÍSTICAS

- **Implantabilidade:** Utiliza o *binding* em tempo de execução para permitir a descoberta e o carregamento das bibliotecas de *plugins* apenas no momento da execução, facilitando a criação de instaladores independentes e a atualização modular do sistema sem a necessidade de recompilar o núcleo.
- **Desempenho (Performance):** Aplica a tática de aumento da eficiência de recursos, por meio de operações *zero-copy* e aceleração de *hardware*, para

reduzir a latência e a sobrecarga do processador, compartilhando referências de texturas de vídeo diretamente na memória da placa gráfica e evitando cópias custosas de pixels brutos.

- **Confiabilidade e Tolerância a Falhas:** Recorre ao isolamento de falhas, à execução assíncrona e ao tratamento local de exceções, com o objetivo de garantir que operações propensas a atrasos ou interrupções externas ocorram em *threads* separadas e que erros imprevistos em um módulo sejam tratados no próprio módulo, impedindo que a falha congele o *loop* principal de orquestração e a interface gráfica ou encerre abruptamente a aplicação.
- **Testabilidade:** Emprega o desacoplamento e a abstração de interfaces para isolar estritamente a lógica de negócio das dependências da interface gráfica (Qt) e dos dispositivos físicos, o que permite criar ambientes de teste nos quais componentes e fontes de áudio e vídeo possam ser facilmente simulados e validados de forma autônoma.
- **Agilidade:** Aplica o encapsulamento e interfaces padronizadas para estabelecer fronteiras rígidas de comunicação, permitindo que a comunidade adicione, modifique e integre novos *plugins* rapidamente, sem risco de corromper o núcleo central.
- **Capacidade de Aprendizado:** Recorre à separação da interface do usuário para manter a lógica de interação visual estritamente na *framework* Qt, com o objetivo de criar painéis de controle intuitivos, o "Modo Estúdio" e *feedbacks* visuais que reduzam a curva de aprendizado para criadores de conteúdo sem conhecimento técnico.
- **Extensibilidade:** Aplica o registro de módulos por meio de uma API padronizada (*obs-module.h*), na qual cada *plugin* declara, na carga (*obs_module_load()*), as fontes, os codificadores e as saídas que oferece, com o objetivo de permitir a inclusão de novas capacidades sem alterar nem recompilar o núcleo, que permanece responsável apenas pela orquestração.

REFERÊNCIAS

MICROSOFT. **Accessibility**. In: Visual Studio Code Documentation. Disponível em: <https://code.visualstudio.com/docs/editor/accessibility>. Acesso em: 29 set. 2026.

MICROSOFT. **Activation Events**. In: Visual Studio Code Extension API. Disponível em: <https://code.visualstudio.com/api/references/activation-events>. Acesso em: 29 set. 2026.

MICROSOFT. **Contribution Points**. In: Visual Studio Code Extension API. Disponível em: <https://code.visualstudio.com/api/references/contribution-points>. Acesso em: 29 set. 2026.

MICROSOFT. **Debug Adapter Protocol**. Especificação. Disponível em: <https://microsoft.github.io/debug-adapter-protocol/>. Acesso em: 29 set. 2026.

MICROSOFT. **Extension Host**. In: Visual Studio Code Extension API. Disponível em: <https://code.visualstudio.com/api/advanced-topics/extension-host>. Acesso em: 29 set. 2026.

MICROSOFT. **Language Server Protocol**. Especificação. Disponível em: <https://microsoft.github.io/language-server-protocol/>. Acesso em: 29 set. 2026.

MICROSOFT. **Source Code Organization**. Wiki do repositório microsoft/vscode. Disponível em: <https://github.com/microsoft/vscode/wiki/Source-Code-Organization>. Acesso em: 29 set. 2026.

MICROSOFT. **Visual Studio Code**. Repositório no GitHub. Disponível em: <https://github.com/microsoft/vscode>. Acesso em: 29 set. 2026.

MICROSOFT. **vscode-test-electron**. Repositório no GitHub. Disponível em: <https://github.com/microsoft/vscode-test-electron>. Acesso em: 29 set. 2026.

MICROSOFT. **Workspace Trust**. In: Visual Studio Code Documentation. Disponível em: <https://code.visualstudio.com/docs/editing/workspace-trust>. Acesso em: 29 set. 2026.

OBS PROJECT. **Backend Design**. In: OBS Studio Documentation. Disponível em: <https://docs.obsproject.com/backend-design>. Acesso em: 29 set. 2026.

OBS PROJECT. **OBS Studio**. Repositório no GitHub. Disponível em: <https://github.com/obsproject/obs-studio>. Acesso em: 29 set. 2026.

OBS PROJECT. **Plugins**. In: OBS Studio Documentation. Disponível em: <https://docs.obsproject.com/plugins>. Acesso em: 29 set. 2026.

PASERO, B.. **Migrating VS Code to Process Sandboxing**. Visual Studio Code Blog, 28 nov. 2022. Disponível em: <https://code.visualstudio.com/blogs/2022/11/28/vscode-sandbox>. Acesso em: 29 set. 2026.

QT GROUP. **Qt Framework**. Site oficial. Disponível em: <https://www.qt.io/>. Acesso em: 29 set. 2026.

RICHARDS, M.; FORD, N. **Fundamentals of Software Architecture: An Engineering Approach**. 1. ed. Sebastopol: O'Reilly Media, 2020.

RIEKEN, J.. **Resolving extension issues with bisect**. Visual Studio Code Blog, 16 fev. 2021. Disponível em: <https://code.visualstudio.com/blogs/2021/02/16/extension-bisect>. Acesso em: 29 set. 2026.